

Resolução da Lista de Exercícios — Programação Linear Inteira

Aluno: Ettore Gabriel Emilio Reis Braga

Disciplina: Programação Linear Inteira

Professor: Prof. Franklin Angelo Krukoski

Data: Julho de 2026

Este notebook integra o relatório da lista de exercícios com o código Python implementado, permitindo a execução interativa e a visualização de todos os resultados.

Ferramentas e Bibliotecas Utilizadas:

- **PuLP + CBC:** resolução das relaxações lineares e modelos inteiros
- **NetworkX + Matplotlib:** construção e visualização de grafos e árvores B&B
- **Pandas:** exibição tabular dos dados e dos passos do Branch-and-Bound

Instalação de dependencias:

pip install pandas matplotlib networkx

```
In [79]: %matplotlib inline
import math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import networkx as nx
import pulp
from collections import deque

# Compatibilidade NumPy >= 2.0 com NetworkX Legado
if not hasattr(np, 'alltrue'):
    np.alltrue = np.all
if not hasattr(np, 'sometrue'):
    np.sometrue = np.any

plt.rcParams.update({'figure.dpi': 110})
EPS = 1e-6
```

Parte 1: Construção da Árvore Branch-and-Bound

Estratégia de busca: busca em profundidade (*depth-first search*, pilha LIFO).

A cada nó resolve-se a **relaxação linear** (variáveis contínuas) com o solver CBC.

Regra de ramificação: escolhe-se a variável inteira de *menor índice* com valor fracionário, gerando dois ramos:

- $x_j \leq \lfloor x_j^* \rfloor$: ramo do *piso*, explorado primeiro
- $x_j \geq \lceil x_j^* \rceil$: ramo do *teto*

Critérios de poda:

1. **Inviabilidade:** a relaxação do nó não tem solução viável
2. **Integralidade:** a solução da relaxação já é inteira (candidata a incumbente)
3. **Limitante** (*bound*): o valor da relaxação não supera o incumbente corrente

Legenda das árvores:

- **Fracionário**
- **Inteiro (incumbente)**
- **Podado por limitante**
- **Inviável**

```
In [80]: # =====
# Motor Branch-and-Bound
# =====

class Node:
    """Representa um nó da árvore Branch-and-Bound."""

    def __init__(
        self,
        node_id: int,
        parent_id: int | None,
        branch_constraint: str,
        variable_bounds: dict,
    ) -> None:
        self.id = node_id
        self.parent = parent_id
        self.branch_txt = branch_constraint
        self.bounds = variable_bounds
        self.step: int | None = None
        self.status: str | None = None
        self.z: float | None = None
        self.sol: dict | None = None
        self.note: str = ''

    def solve_relaxation(
        prob_data: dict,
        variable_bounds: dict,
    ) -> tuple[float | None, dict | None]:
        """Resolve a relaxação linear com limites em `variable_bounds`."""
        sense = prob_data['sense']
        var_names = prob_data['vars']
        obj_coefs = prob_data['obj']
        constraints = prob_data['constraints']

        model = pulp.LpProblem('relax', sense)
```

```

decision_vars: dict = {}
for name in var_names:
    lo, hi = variable_bounds.get(name, (0, None))
    decision_vars[name] = pulp.LpVariable(
        name, lowBound=lo, upBound=hi, cat='Continuous'
    )

model += pulp.lpSum(
    obj_coefs.get(name, 0) * decision_vars[name]
    for name in var_names
)
for coef_dict, sign, rhs in constraints:
    expr = pulp.lpSum(
        coef_dict.get(name, 0) * decision_vars[name]
        for name in var_names
    )
    if sign == '<=':
        model += expr <= rhs
    elif sign == '>=':
        model += expr >= rhs
    else:
        model += expr == rhs

lp_status = model.solve(pulp.PULP_CBC_CMD(msg=0))
if pulp.LpStatus[lp_status] != 'Optimal':
    return None, None
solution = {
    name: decision_vars[name].value() for name in var_names
}
return pulp.value(model.objective), solution

def find_first_fractional(
    solution: dict,
    integer_vars: list,
) -> tuple[str | None, float | None]:
    """Retorna (nome, valor) da 1ª variável inteira fracionária."""
    for var_name in integer_vars:
        value = solution[var_name]
        if abs(value - round(value)) > 1e-4:
            return var_name, value
    return None, None

def branch_and_bound(
    prob_data: dict,
    max_steps: int = 500,
) -> dict:
    """Executa B&B com busca em profundidade (pilha LIFO)."""
    sense = prob_data['sense']
    integer_vars = prob_data['int_vars']
    is_maximize = (sense == pulp.LpMaximize)

    incumbent_z: float | None = None
    incumbent_sol: dict | None = None
    all_nodes: list = []
    id_counter = [0]

    def create_node(
        parent_id: int | None,

```

```

        branch_constraint: str,
        bounds: dict,
    ) -> Node:
        node = Node(
            id_counter[0], parent_id,
            branch_constraint, bounds,
        )
        id_counter[0] += 1
        all_nodes.append(node)
        return node

root = create_node(None, 'raiz (LP)', {})
stack = [root]
steps_done = 0

while stack and steps_done < max_steps:
    node = stack.pop()
    steps_done += 1
    node.step = steps_done
    obj_val, sol = solve_relaxation(prob_data, node.bounds)

    if obj_val is None:
        node.status = 'inviavel'
        node.note = 'Poda por inviabilidade'
        continue

    node.z, node.sol = obj_val, sol

    # poda por limitante
    if incumbent_z is not None:
        no_improvement = (
            is_maximize and obj_val <= incumbent_z + EPS
        ) or (
            not is_maximize and obj_val >= incumbent_z - EPS
        )
        if no_improvement:
            node.status = 'podado'
            node.note = (
                f'Poda: z={obj_val:.3f} nao melhora '
                f'incumbente {incumbent_z:.3f}'
            )
            continue

    branch_var, branch_val = find_first_fractional(
        sol, integer_vars
    )

    if branch_var is None:
        node.status = 'inteiro'
        is_better = (
            incumbent_z is None
            or (is_maximize and obj_val > incumbent_z)
            or (not is_maximize and obj_val < incumbent_z)
        )
        if is_better:
            incumbent_z = obj_val
            incumbent_sol = dict(sol)
            node.note = 'Nova solucao incumbente'
        else:
            node.note = (

```



```

        'Solucao inteira '
        '(nao melhora incumbente)'
    )
    continue

    node.status = 'fracionario'
    floor_val = math.floor(branch_val)
    ceil_val = math.ceil(branch_val)

    bounds_floor = dict(node.bounds)
    lo, hi = bounds_floor.get(branch_var, (0, None))
    bounds_floor[branch_var] = (lo, floor_val)
    child_floor = create_node(
        node.id,
        f'{branch_var} <= {floor_val}',
        bounds_floor,
    )

    bounds_ceil = dict(node.bounds)
    lo, hi = bounds_ceil.get(branch_var, (0, None))
    bounds_ceil[branch_var] = (ceil_val, hi)
    child_ceil = create_node(
        node.id,
        f'{branch_var} >= {ceil_val}',
        bounds_ceil,
    )

    node.note = (
        f'Ramifica em {branch_var} = {branch_val:.4f}'
    )
    # TETO por último -> PISO sai primeiro (pilha LIFO)
    stack.append(child_ceil)
    stack.append(child_floor)

    return {
        'nodes': all_nodes,
        'incumbent_z': incumbent_z,
        'incumbent_sol': incumbent_sol,
    }

print('Motor B&B carregado.')

```

Motor B&B carregado.

```

In [81]: # =====
# Utilitários de visualização da árvore B&B
# =====
STATUS_COLORS: dict = {
    'inteiro': '#2ca02c',
    'fracionario': '#1f77b4',
    'inviavel': '#d62728',
    'podado': '#ff7f0e',
    None: '#999999',
}

def format_solution(solution: dict | None) -> str:
    """Formata a solução como string compacta."""
    if solution is None:

```

```

        return '--'
    return ', '.join(
        f'{var}={val:.3f}'.rstrip('0').rstrip('.')
        for var, val in solution.items()
    )

def compute_tree_layout(nodes: list) -> dict:
    """Calcula posições (x, y) de cada nó para desenho."""
    children_map: dict = {node.id: [] for node in nodes}
    root_id: int | None = None
    for node in nodes:
        if node.parent is None:
            root_id = node.id
        else:
            children_map[node.parent].append(node.id)

    depth_map: dict = {}

    def set_depth(node_id: int, current_depth: int) -> None:
        depth_map[node_id] = current_depth
        for child_id in children_map[node_id]:
            set_depth(child_id, current_depth + 1)

    set_depth(root_id, 0)

    x_positions: dict = {}
    leaf_counter = [0]

    def assign_x(node_id: int) -> None:
        child_ids = children_map[node_id]
        if not child_ids:
            x_positions[node_id] = leaf_counter[0]
            leaf_counter[0] += 1
        else:
            for child_id in child_ids:
                assign_x(child_id)
            x_positions[node_id] = (
                sum(x_positions[c] for c in child_ids)
                / len(child_ids)
            )

    assign_x(root_id)
    return {
        node.id: (x_positions[node.id], -depth_map[node.id])
        for node in nodes
    }

def draw_tree(res: dict, title: str = 'Árvore B&B') -> None:
    """Desenha a árvore B&B com nós coloridos por status."""
    nodes = res['nodes']
    positions = compute_tree_layout(nodes)

    graph = nx.DiGraph()
    for node in nodes:
        graph.add_node(node.id)
        if node.parent is not None:
            graph.add_edge(node.parent, node.id)

```

```

fig_width = max(8, len(nodes) * 1.05)
fig, ax = plt.subplots(figsize=(fig_width, 6))
nx.draw_networkx_edges(
    graph, positions, ax=ax,
    edge_color='#888', arrows=False,
)
for node in nodes:
    x_pos, y_pos = positions[node.id]
    color = STATUS_COLORS.get(node.status, '#999999')
    ax.scatter(
        [x_pos], [y_pos], s=1500, c=color,
        zorder=3, edgecolors='black',
    )
    node_label = f'N{node.id}'
    if node.step:
        node_label += f'\n(p{node.step})'
    ax.text(
        x_pos, y_pos, node_label,
        ha='center', va='center',
        color='white', fontsize=8,
        fontweight='bold', zorder=4,
    )
    if node.status == 'inviavel':
        info_text = 'inviavel'
    elif node.z is not None:
        info_text = (
            f'z={node.z:.2f}\n'
            f'{format_solution(node.sol)}'
        )
    else:
        info_text = '(nao explorado)'
    ax.text(
        x_pos, y_pos - 0.32, info_text,
        ha='center', va='top', fontsize=7,
        bbox=dict(
            boxstyle='round,pad=0.2',
            fc='#f5f5f5', ec='#ccc',
        ),
    )
    if node.parent is not None:
        px, py = positions[node.parent]
        label_x = px + 0.70 * (x_pos - px)
        label_y = py + 0.70 * (y_pos - py) + 0.13
        ax.text(
            label_x, label_y, node.branch_txt,
            fontsize=7.5, color='#1a1a1a',
            ha='center', fontweight='bold',
            bbox=dict(
                boxstyle='round,pad=0.15',
                fc='#fff7e0', ec='#e0c060',
            ),
        )

legend_patches = [
    mpatches.Patch(color='#1f77b4', label='Fracionario'),
    mpatches.Patch(
        color='#2ca02c', label='Inteiro (incumbente)'
    ),
    mpatches.Patch(
        color='#ff7f0e', label='Podado por limitante'
    )
]

```

```

    ),
    mpatches.Patch(color='#d62728', label='Inviavel'),
]
ax.legend(
    handles=legend_patches, loc='upper right', fontsize=8
)
ax.set_title(title, fontsize=12, fontweight='bold')
ax.axis('off')
plt.tight_layout()
plt.show()

def results_table(res: dict) -> pd.DataFrame:
    """Converte os passos do B&B em um DataFrame tabular."""
    explored_nodes = sorted(
        [node for node in res['nodes'] if node.step],
        key=lambda node: node.step,
    )
    rows = []
    for node in explored_nodes:
        z_val = f'{node.z:.3f}' if node.z is not None else '--'
        rows.append({
            'Passo': node.step,
            'No': f'N{node.id}',
            'Ramo': node.branch_txt,
            'z_LP': z_val,
            'Solucao LP': format_solution(node.sol),
            'Status': node.status,
            'Observacao': node.note,
        })
    return pd.DataFrame(rows)

print('Utilitarios de visualizacao carregados.')

```

Utilitarios de visualizacao carregados.

Exercício 1

$$\max z = x_1 + 5x_2 + 9x_3 + 5x_4$$

Sujeito a:

- $x_1 + 3x_2 + 9x_3 + 6x_4 \leq 16$
- $6x_1 + 6x_2 + 7x_4 \leq 19$
- $7x_1 + 8x_2 + 18x_3 + 3x_4 \leq 44$
- $x_1, x_2, x_3, x_4 \geq 0$ e inteiras

A relaxação na raiz fornece $z = 22,33$ (fracionária). A busca em profundidade explora **13** nós.

```

In [82]: prob_ex1 = {
    'name': 'Exercício 1',
    'sense': pulp.LpMaximize,
    'vars': ['x1', 'x2', 'x3', 'x4'],
    'int_vars': ['x1', 'x2', 'x3', 'x4'],
    'obj': {'x1': 1, 'x2': 5, 'x3': 9, 'x4': 5},
    'constraints': [

```

```

        ({'x1': 1, 'x2': 3, 'x3': 9, 'x4': 6}, '<=', 16),
        ({'x1': 6, 'x2': 6, 'x4': 7}, '<=', 19),
        ({'x1': 7, 'x2': 8, 'x3': 18, 'x4': 3}, '<=', 44),
    ],
}

res1 = branch_and_bound(prob_ex1)
print(f"z* = {res1['incumbent_z']} | {res1['incumbent_sol']}")
print(f"Nós explorados: {len([n for n in res1['nodes'] if n.step])}")
results_table(res1)

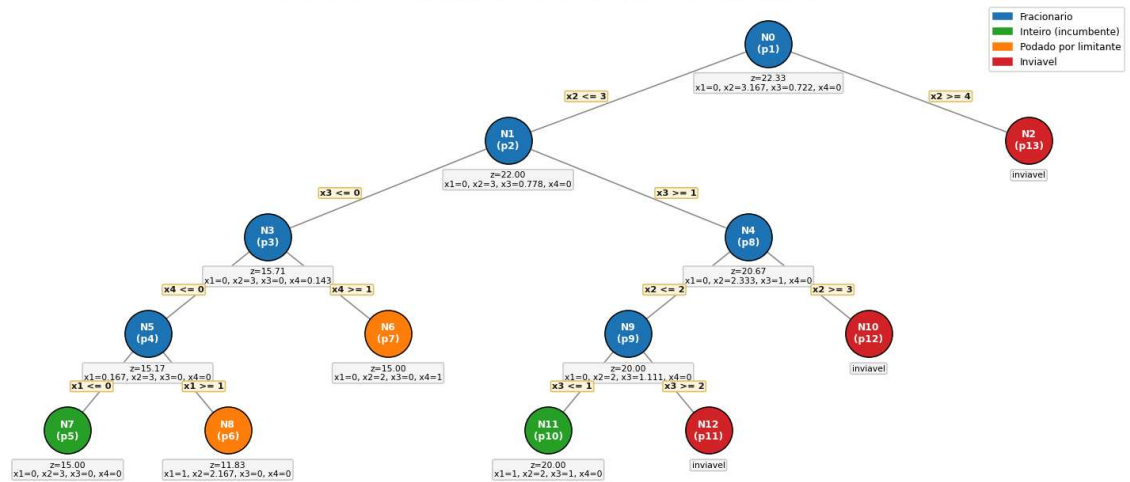
```

z* = 20.0 | {'x1': 1.0, 'x2': 2.0, 'x3': 1.0, 'x4': 0.0}
Nós explorados: 13

Out[82]:	Passo	No	Ramo	z_LP	Solucao LP	Status	Observacao
0	1	N0	raiz (LP)	22.333	x1=0, x2=3.167, x3=0.722, x4=0	fracionario	Ramifica em x2 = 3.1667
1	2	N1	x2 <= 3	22.000	x1=0, x2=3, x3=0.778, x4=0	fracionario	Ramifica em x3 = 0.7778
2	3	N3	x3 <= 0	15.714	x1=0, x2=3, x3=0, x4=0.143	fracionario	Ramifica em x4 = 0.1429
3	4	N5	x4 <= 0	15.167	x1=0.167, x2=3, x3=0, x4=0	fracionario	Ramifica em x1 = 0.1667
4	5	N7	x1 <= 0	15.000	x1=0, x2=3, x3=0, x4=0	inteiro	Nova solucao incumbente
5	6	N8	x1 >= 1	11.833	x1=1, x2=2.167, x3=0, x4=0	podado	Poda: z=11.833 nao melhora incumbente 15.000
6	7	N6	x4 >= 1	15.000	x1=0, x2=2, x3=0, x4=1	podado	Poda: z=15.000 nao melhora incumbente 15.000
7	8	N4	x3 >= 1	20.667	x1=0, x2=2.333, x3=1, x4=0	fracionario	Ramifica em x2 = 2.3333
8	9	N9	x2 <= 2	20.000	x1=0, x2=2, x3=1.111, x4=0	fracionario	Ramifica em x3 = 1.1111
9	10	N11	x3 <= 1	20.000	x1=1, x2=2, x3=1, x4=0	inteiro	Nova solucao incumbente
10	11	N12	x3 >= 2	--	--	inviavel	Poda por inviabilidade
11	12	N10	x2 >= 3	--	--	inviavel	Poda por inviabilidade
12	13	N2	x2 >= 4	--	--	inviavel	Poda por inviabilidade

In [83]: draw_tree(res1, 'Árvore B&B – Exercício 1 (z* = 20, ótimo: x₁=1, x₂=2, x₃=1, x

Árvore B&B — Exercício 1 ($z^* = 20$, ótimo: $x_1=1, x_2=2, x_3=1, x_4=0$)



Exercício 2

$$\max z = 7x_1 + 9x_2 + x_3 + 6x_4$$

Sujeito a:

- $8x_1 + 2x_2 + 4x_3 + 2x_4 \leq 16$
- $4x_1 + 8x_2 + 2x_3 \leq 20$
- $7x_1 + 6x_3 + 2x_4 \leq 11$
- $x_1, x_2, x_4 \geq 0$ inteiras; $x_3 \geq 0$ contínua

Como x_3 é contínua, a ramificação incide apenas sobre x_1, x_2, x_4 . A árvore completa tem **7 nós**.

```
In [84]: prob_ex2 = {
    'name': 'Exercício 2',
    'sense': pulp.LpMaximize,
    'vars': ['x1', 'x2', 'x3', 'x4'],
    'int_vars': ['x1', 'x2', 'x4'], # x3 é contínua
    'obj': {'x1': 7, 'x2': 9, 'x3': 1, 'x4': 6},
    'constraints': [
        ({'x1': 8, 'x2': 2, 'x3': 4, 'x4': 2}, '<=', 16),
        ({'x1': 4, 'x2': 8, 'x3': 2}, '<=', 20),
        ({'x1': 7, 'x3': 6, 'x4': 2}, '<=', 11),
    ],
}

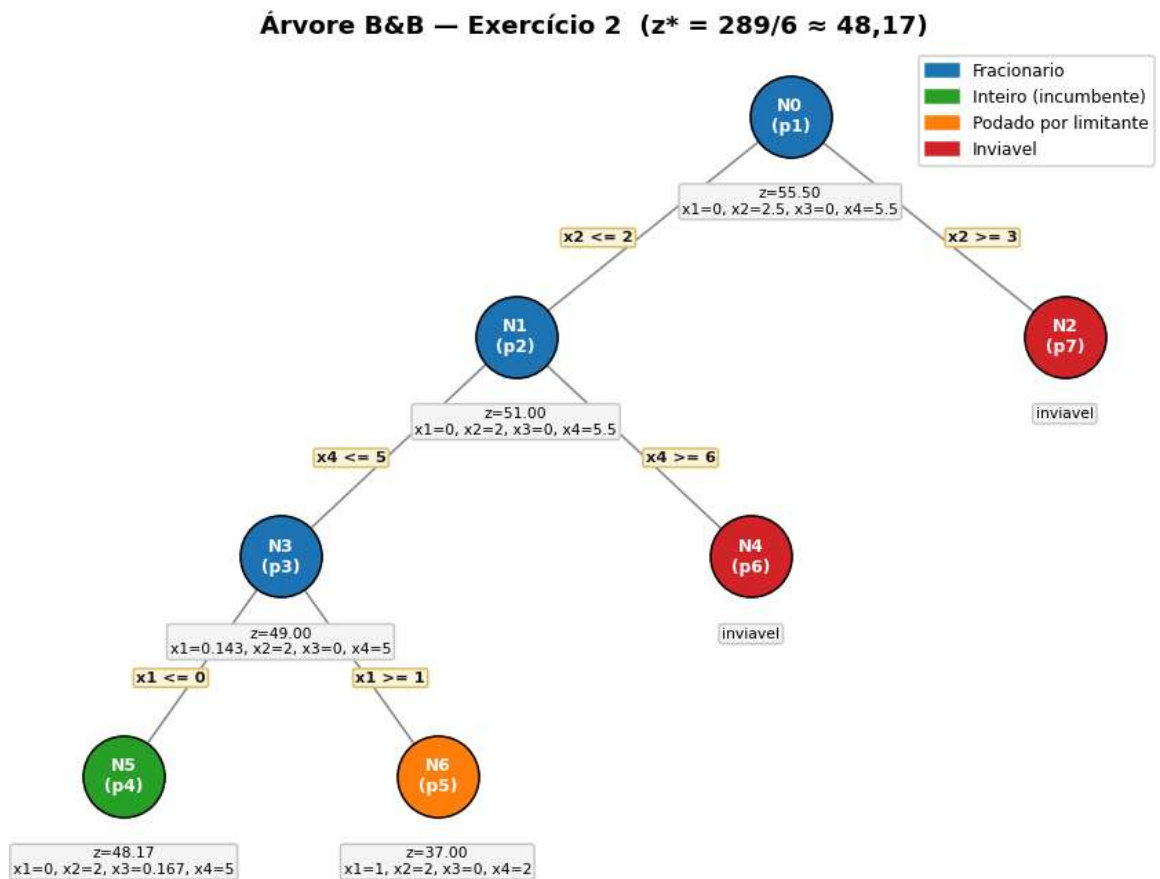
res2 = branch_and_bound(prob_ex2)
print(f"z* = {res2['incumbent_z']:.4f} | {res2['incumbent_sol']}")
results_table(res2)
```

$z^* = 48.1667$ | $\{x_1: 0.0, x_2: 2.0, x_3: 0.16666667, x_4: 5.0\}$

Out[84]:

	Passo	No	Ramo	z_LP	Solucao LP	Status	Observacao
0	1	N0	raiz (LP)	55.500	x1=0, x2=2.5, x3=0, x4=5.5	fracionario	Ramifica em x2 = 2.5000
1	2	N1	x2 ≤ 2	51.000	x1=0, x2=2, x3=0, x4=5.5	fracionario	Ramifica em x4 = 5.5000
2	3	N3	x4 ≤ 5	49.000	x1=0.143, x2=2, x3=0, x4=5	fracionario	Ramifica em x1 = 0.1429
3	4	N5	x1 ≤ 0	48.167	x1=0, x2=2, x3=0.167, x4=5	inteiro	Nova solucao incumbente
4	5	N6	x1 ≥ 1	37.000	x1=1, x2=2, x3=0, x4=2	podado	Poda: z=37.000 nao melhora incumbente 48.167
5	6	N4	x4 ≥ 6	--	--	inviavel	Poda por inviabilidade
6	7	N2	x2 ≥ 3	--	--	inviavel	Poda por inviabilidade

In [85]: draw_tree(res2, 'Árvore B&B — Exercício 2 (z* = 289/6 ≈ 48,17)')



Exercício 3

$$\min z = 3x_1 + 4x_2 + 3x_3$$

Sujeito a:

- $3x_1 + 2x_2 + 2x_3 \geq 13$
- $2x_1 + 5x_2 + 3x_3 \geq 15$
- $2x_1 + x_2 + 2x_3 \geq 9$
- $x_2, x_3 \geq 0$ **inteiras**; $x_1 \geq 0$ **contínua**

x_1 é contínua; ramifica-se sobre x_2, x_3 . A árvore completa tem **9 nós**.

```
In [86]: prob_ex3 = {
    'name':      'Exercício 3',
    'sense':     pulp.LpMinimize,
    'vars':      ['x1', 'x2', 'x3'],
    'int_vars':  ['x2', 'x3'],    # x1 é contínua
    'obj':       {'x1': 3, 'x2': 4, 'x3': 3},
    'constraints': [
        ({'x1': 3, 'x2': 2, 'x3': 2}, '>=', 13),
        ({'x1': 2, 'x2': 5, 'x3': 3}, '>=', 15),
        ({'x1': 2, 'x2': 1, 'x3': 2}, '>=', 9),
    ],
}

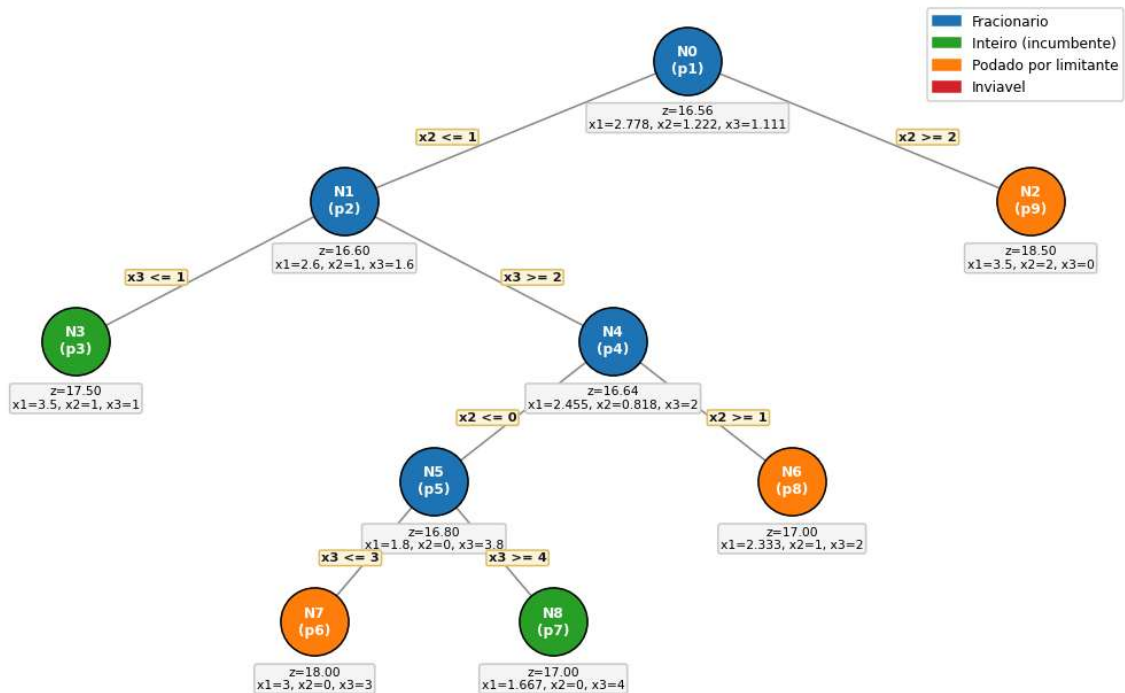
res3 = branch_and_bound(prob_ex3)
print(f"z* = {res3['incumbent_z']} | {res3['incumbent_sol']}")
results_table(res3)
```

z* = 17.0000001 | {'x1': 1.6666667, 'x2': 0.0, 'x3': 4.0}

Out[86]:	Passo	No	Ramo	z_LP	Solucao LP	Status	Observacao
0	1	N0	raiz (LP)	16.556	x1=2.778, x2=1.222, x3=1.111	fracionario	Ramifica em x2 = 1.2222
1	2	N1	x2 <= 1	16.600	x1=2.6, x2=1, x3=1.6	fracionario	Ramifica em x3 = 1.6000
2	3	N3	x3 <= 1	17.500	x1=3.5, x2=1, x3=1	inteiro	Nova solucao incumbente
3	4	N4	x3 >= 2	16.636	x1=2.455, x2=0.818, x3=2	fracionario	Ramifica em x2 = 0.8182
4	5	N5	x2 <= 0	16.800	x1=1.8, x2=0, x3=3.8	fracionario	Ramifica em x3 = 3.8000
5	6	N7	x3 <= 3	18.000	x1=3, x2=0, x3=3	podado	Poda: z=18.000 nao melhora incumbente 17.500
6	7	N8	x3 >= 4	17.000	x1=1.667, x2=0, x3=4	inteiro	Nova solucao incumbente
7	8	N6	x2 >= 1	17.000	x1=2.333, x2=1, x3=2	podado	Poda: z=17.000 nao melhora incumbente 17.000
8	9	N2	x2 >= 2	18.500	x1=3.5, x2=2, x3=0	podado	Poda: z=18.500 nao melhora incumbente 17.000

```
In [87]: draw_tree(res3, 'Árvore B&B – Exercício 3 (z* = 17, ótimo: x1=5/3, x2=0, x3=4)')
```


Árvore B&B — Exercício 3 ($z^* = 17$, ótimo: $x_1=5/3$, $x_2=0$, $x_3=4$)



Exercício 4

$$\min z = 2x_1 + 3x_2 + 5x_3$$

Sujeito a:

- $x_1 + 2x_2 + 3x_3 \geq 7$
- $3x_1 + 2x_2 + 3x_3 \geq 11$
- $x_1, x_3 \geq 0$ inteiras; $x_2 \geq 0$ contínua

A relaxação na raiz já fornece $x_1 = 2$, $x_3 = 0$ (inteiros) com $x_2 = 2,5$ (contínua). A árvore se encerra na raiz **sem ramificação**.

```
In [88]: prob_ex4 = {
    'name': 'Exercício 4',
    'sense': pulp.LpMinimize,
    'vars': ['x1', 'x2', 'x3'],
    'int_vars': ['x1', 'x3'], # x2 é contínua
    'obj': {'x1': 2, 'x2': 3, 'x3': 5},
    'constraints': [
        ({'x1': 1, 'x2': 2, 'x3': 3}, '>=', 7),
        ({'x1': 3, 'x2': 2, 'x3': 3}, '>=', 11),
    ],
}

res4 = branch_and_bound(prob_ex4)
print(f"z* = {res4['incumbent_z']} | {res4['incumbent_sol']}")
print('A árvore se encerra na raiz (sem ramificação): relaxação LP já é inteira')
results_table(res4)
```

$z^* = 11.5$ | $\{x_1: 2.0, x_2: 2.5, x_3: 0.0\}$

A árvore se encerra na raiz (sem ramificação): relaxação LP já é inteira nas variáveis exigidas.

Out[88]:

	Passo	No	Ramo	z_LP	Solucao LP	Status	Observacao
0	1	N0	raiz (LP)	11.500	x1=2, x2=2.5, x3=0	inteiro	Nova solucao incumbente

Parte 2: Formulação de Problemas de PLI

Apresentam-se apenas os modelos matemáticos (variáveis de decisão, função objetivo e restrições), conforme solicitado. Não há código de execução nesta parte.

Exercício 1: Carregamento do avião cargueiro

Variáveis: $n_{ik} \in \mathbb{Z}_{\geq 0}$ = contêineres do tipo $i \in \{1, 2, 3\}$ no compartimento $k \in \{F, C, T\}$; $g_j \geq 0$ = carga a granel $j \in \{4, 5\}$ (ton) no porão G .

Objetivo (maximizar lucro):

$$\max z = \sum_k (200 w_1 n_{1k} + 220 w_2 n_{2k} + 175 w_3 n_{3k}) + 235 g_4 + 180 g_5$$

Restrições de peso (ton: $F = 5, C = 7, T = 6, G = 7$):

$$\sum_i w_i n_{iF} \leq 5, \quad \sum_i w_i n_{iC} \leq 7, \quad \sum_i w_i n_{iT} \leq 6, \quad g_4 + g_5 \leq 7$$

Equilíbrio de voo:

$$\frac{\sum_i w_i n_{iF}}{5} = \frac{\sum_i w_i n_{iC}}{7} = \frac{\sum_i w_i n_{iT}}{6} = \frac{g_4 + g_5}{7}$$

Exercício 2: Combate ao incêndio florestal

Variáveis: $x_1, x_2, x_3 \in \mathbb{Z}_{\geq 0}$ = helicópteros AH-1, aviões-tanque e aviões B67.

$$\min z = 6000x_1 + 12000x_2 + 30000x_3$$

- $3(15000x_1 + 40000x_2 + 85000x_3) \geq 3\,000\,000$ -> área coberta
- $2x_1 \leq 10$ -> pilotos de helicóptero
- $2x_2 + 2x_3 \leq 14$ -> pilotos de avião
- $x_2 + 3x_3 \leq 22$ -> operadores

Exercício 4: Redução de peso do automóvel

Variáveis: $x_j \in \{0, 1\}$, $j = 1, \dots, 12$ (1 se a alteração j for implementada).

$$\min z = \sum_{j=1}^{12} c_j x_j \quad \text{s.a.} \quad \sum_{j=1}^{12} r_j x_j \geq 180$$

Reduções $r = (30, 20, 25, 40, 15, 10, 60, 80, 40, 30, 50, 35)$ kg

Custos $c = (130, 110, 120, 150, 80, 80, 360, 400, 160, 120, 200, 160)$ (mil \$)

Exercício 5: Localização de Centros de Distribuição

Variáveis: $y_i \in \{0, 1\}$ = 1 se o CD i é instalado; $x_{ij} \geq 0$ = quantidade enviada de i a j .

$$\min z = \sum_i f_i y_i + \sum_i \sum_j (v_i + c_{ij}) x_{ij}$$

- $\sum_i x_{ij} = d_j$ para todo j -> atender demanda
- $\sum_j x_{ij} \leq \text{Cap}_i y_i$ para todo i -> capacidade e abertura (só despacha de CD aberto)

Parte 3: Transporte, Transbordo e Designação

Os problemas de transporte são modelados como:

$$\min \sum_{ij} c_{ij} x_{ij} \quad \text{s.a.} \quad \sum_j x_{ij} \leq s_i \text{ (oferta)}, \quad \sum_i x_{ij} = d_j \text{ (demanda)}, \quad x_{ij} \geq 0$$

Em todos os casos a oferta total é maior ou igual à demanda total. O excesso é absorvido pela folga da oferta.

O problema de designação (Ex. 5) usa $x_{ij} \in \{0, 1\}$ com $\sum_j x_{ij} = 1$ e $\sum_i x_{ij} = 1$.

```
In [89]: # =====  
# Funções de resolução: transporte e designação  
# =====  
def solve_transport(  
    cost_matrix: list,  
    supply: list,  
    demand: list,  
) -> tuple:  
    """Resolve o problema de transporte por PL continua."""  
    n_sources = len(supply)  
    n_destinations = len(demand)  
    model = pulp.LpProblem('transp', pulp.LpMinimize)  
  
    flow_vars = [  
        [  
            pulp.LpVariable(f'x_{i}_{j}', lowBound=0)  
            for j in range(n_destinations)  
        ]  
        for i in range(n_sources)  
    ]  
  
    model += pulp.lpSum(  
        cost_matrix[i][j] * flow_vars[i][j]  
        for i in range(n_sources)  
        for j in range(n_destinations)  
    )  
    for i in range(n_sources):  
        model += (  
            pulp.lpSum(  
                flow_vars[i][j]  
                for j in range(n_destinations)
```

```

        ) <= supply[i]
    )
    for j in range(n_destinations):
        model += (
            pulp.lpSum(
                flow_vars[i][j]
                for i in range(n_sources)
            ) == demand[j]
        )

    model.solve(pulp.PULP_CBC_CMD(msg=0))
    allocation = np.array([
        [
            flow_vars[i][j].value()
            for j in range(n_destinations)
        ]
        for i in range(n_sources)
    ])
    return allocation, pulp.value(model.objective)

def solve_assignment(cost_matrix: list) -> tuple:
    """Resolve o problema de designação com variáveis binárias."""
    n_workers = len(cost_matrix)
    model = pulp.LpProblem('assign', pulp.LpMinimize)

    assign_vars = [
        [
            pulp.LpVariable(f'x_{i}_{j}', cat='Binary')
            for j in range(n_workers)
        ]
        for i in range(n_workers)
    ]

    model += pulp.lpSum(
        cost_matrix[i][j] * assign_vars[i][j]
        for i in range(n_workers)
        for j in range(n_workers)
    )

    for i in range(n_workers):
        model += (
            pulp.lpSum(
                assign_vars[i][j] for j in range(n_workers)
            ) == 1
        )

    for j in range(n_workers):
        model += (
            pulp.lpSum(
                assign_vars[i][j] for i in range(n_workers)
            ) == 1
        )

    model.solve(pulp.PULP_CBC_CMD(msg=0))
    result_matrix = np.array([
        [
            int(round(assign_vars[i][j].value()))
            for j in range(n_workers)
        ]
        for i in range(n_workers)
    ])

```

```

return result_matrix, pulp.value(model.objective)

def show_transport_matrix(
    matrix: np.ndarray,
    rows: list,
    cols: list,
    title: str,
    fmt: str = '{:.0f}',
) -> None:
    """Exibe a matriz de alocação com rotas ativas em verde."""
    n_rows = len(rows)
    n_cols = len(cols)
    fig_w = 1.5 + 0.9 * n_cols
    fig_h = 1.0 + 0.55 * n_rows
    fig, ax = plt.subplots(figsize=(fig_w, fig_h))
    ax.axis('off')

    cell_data = [
        [
            fmt.format(matrix[i][j])
            if matrix[i][j] > 1e-6 else '-'
            for j in range(n_cols)
        ]
        for i in range(n_rows)
    ]
    table = ax.table(
        cellText=cell_data,
        rowLabels=rows,
        colLabels=cols,
        cellLoc='center',
        loc='center',
    )
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.scale(1, 1.5)
    for i in range(n_rows):
        for j in range(n_cols):
            if matrix[i][j] > 1e-6:
                table[(i + 1, j)].set_facecolor('#cfe8cf')
    ax.set_title(title, fontweight='bold', pad=6)
    plt.tight_layout()
    plt.show()

print('Funcoes de transporte e designacao carregadas.')

```

Funcoes de transporte e designacao carregadas.

Exercício 1: Transporte (3 fábricas × 4 mercados)

Oferta (220, 180, 230), demanda (150, 165, 210, 90); oferta total 630 > 615.

	M1	M2	M3	M4
F1	10	7	5	6
F2	12	7	6	4
F3	13	6	3	5

```
In [90]: cost1 = [[10, 7, 5, 6],
                 [12, 7, 6, 4],
                 [13, 6, 3, 5]]
sup1    = [220, 180, 230]
dem1    = [150, 165, 210, 90]

X1, z1 = solve_transport(cost1, sup1, dem1)
print(f'Custo mínimo: R$ {z1:,.2f}')
show_transport_matrix(X1,
                      rows=['Fab.1', 'Fab.2', 'Fab.3'],
                      cols=['Merc.1', 'Merc.2', 'Merc.3', 'Merc.4'],
                      title=f'Ex. 1 – Plano ótimo de transporte (custo = R$ {z1:.0f})')
```

Custo mínimo: R\$ 3,625.00

Ex. 1 — Plano ótimo de transporte (custo = R\$ 3625)

	Merc.1	Merc.2	Merc.3	Merc.4
Fab.1	150	70	-	-
Fab.2	-	75	-	90
Fab.3	-	20	210	-

Exercício 2: Compra de pneus (Expresso Rápido Faxina)

Modelado como transporte: ofertas = pneus por revendedor

($A = 12000, B = 6000, C = 4000$); demandas = pneus por terminal

(4000, 8000, 3000, 5000).

```
In [91]: # Custo[revendedor][terminal]
cost2 = [[70, 74, 62, 62],
         [64, 62, 68, 72],
         [68, 65, 64, 66]]
sup2   = [12000, 6000, 4000]
dem2   = [4000, 8000, 3000, 5000]

X2, z2 = solve_transport(cost2, sup2, dem2)
print(f'Custo mínimo: R$ {z2:,.2f}')
show_transport_matrix(X2,
                      rows=['Rev.A', 'Rev.B', 'Rev.C'],
                      cols=['Curitiba', 'Londrina', 'Cascavel', 'C.Mourão'],
                      title=f'Ex. 2 – Compras ótimas de pneus (custo = R$ {z2:.0f})')
```

Custo mínimo: R\$ 1,272,000.00

Ex. 2 — Compras ótimas de pneus (custo = R\$ 1272000)

	Curitiba	Londrina	Cascavel	C.Mourão
Rev.A	2000	-	3000	5000
Rev.B	2000	4000	-	-
Rev.C	-	4000	-	-

Exercício 3: Aquisição de combustível

Ofertas (mil galões) (320, 270, 150); demandas (100, 180, 300).

```
In [92]: cost3 = [[92, 89, 90],
                 [91, 91, 95],
                 [87, 90, 92]]
sup3 = [320, 270, 150]
dem3 = [100, 180, 300]

X3, z3 = solve_transport(cost3, sup3, dem3)
print(f'Custo mínimo: US$ {z3 * 1000:,.0f}')
show_transport_matrix(X3,
                      rows=['Forn.1', 'Forn.2', 'Forn.3'],
                      cols=['Aerop.1', 'Aerop.2', 'Aerop.3'],
                      title=f'Ex. 3 — Política ótima de aquisição (mil galões) (z = {z3:.0f})')
```

Custo mínimo: US\$ 51,990,000

Ex. 3 — Política ótima de aquisição (mil galões) (z = 51990)

	Aerop.1	Aerop.2	Aerop.3
Forn.1	-	20	300
Forn.2	-	110	-
Forn.3	100	50	-

Exercício 4: Mercados "Deise-Luzia"

Três centros C_1, C_2, C_3 (capacidades 500, 750, 400) atendem 11 armazéns. Custo $c_{ij} = 0,50 \cdot d_{ij}$ (\$/ton).

```
In [93]: dist4 = [
    [10, 22, 29, 45, 11, 31, 42, 61, 36, 21, 45],
    [25, 35, 17, 38, 9, 17, 65, 45, 42, 5, 41],
    [18, 19, 22, 29, 24, 54, 39, 78, 51, 14, 38],
]
cost4 = [[0.5 * d for d in row] for row in dist4]
cap4 = [500, 750, 400]
dem4 = [112, 85, 138, 146, 77, 89, 101, 215, 53, 49, 153]

X4, z4 = solve_transport(cost4, cap4, dem4)
print(f'Custo mínimo: $ {z4:,.2f}')
show_transport_matrix(X4,
    rows=['C1', 'C2', 'C3'],
    cols=[f'W{j+1}' for j in range(11)],
    title=f'Ex. 4 – Programa ótimo (ton) (custo = $ {z4:,.2f})')
```

Custo mínimo: \$ 16,678.50

Ex. 4 – Programa ótimo (ton) (custo = \$ 16678.50)

	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11
C1	112	-	-	-	-	-	-	-	53	-	-
C2	-	-	138	-	77	89	-	215	-	49	85
C3	-	85	-	146	-	-	101	-	-	-	68

Parte 4: Mínima Arborescência e Fluxo Máximo

Os pesos/capacidades foram transcritos das figuras do enunciado. Os algoritmos foram implementados em Python.

Mínima Árvore Geradora: dado um grafo não-dirigido $G = (V, E, w)$, encontrar $T \subseteq E$ que conecta todos os vértices com $\sum_{e \in T} w_e$ mínimo.

Fluxo Máximo (Edmonds-Karp): algoritmo de Ford-Fulkerson usando BFS no grafo residual para encontrar caminhos aumentantes (complexidade $O(VE^2)$).

```
In [94]: # =====
# Algoritmos: Kruskal e Prim (Árvore Geradora Mínima)
# =====
def kruskal(nodes: list, edges: list) -> tuple:
    """Kruskal com Union-Find. Retorna (arvore, custo, log)."""
    parent: dict = {node: node for node in nodes}

    def find_root(vertex: int) -> int:
        while parent[vertex] != vertex:
            parent[vertex] = parent[parent[vertex]]
            vertex = parent[vertex]
        return vertex

    step_log: list = []
    tree_edges: list = []
    total_cost: int = 0

    for u, v, weight in sorted(edges, key=lambda e: e[2]):
```



```

    root_u, root_v = find_root(u), find_root(v)
    if root_u != root_v:
        parent[root_u] = root_v
        tree_edges.append((u, v, weight))
        total_cost += weight
        step_log.append(
            (u, v, weight, 'ACEITA',
             f'une {{{u}}} e {{{v}}}')
        )
    else:
        step_log.append(
            (u, v, weight, 'rejeita', 'formaria ciclo')
        )
    return tree_edges, total_cost, step_log

def prim(nodes: list, edges: list, start: int) -> tuple:
    """Prim guloso a partir de `start`.
    Retorna (arvore, custo, log).
    """
    adjacency: dict = {node: [] for node in nodes}
    for u, v, weight in edges:
        adjacency[u].append((weight, v))
        adjacency[v].append((weight, u))

    visited_nodes = {start}
    step_log: list = []
    tree_edges: list = []
    total_cost: int = 0

    while len(visited_nodes) < len(nodes):
        best_edge = None
        for u in visited_nodes:
            for weight, v in adjacency[u]:
                if v not in visited_nodes:
                    if (best_edge is None
                        or weight < best_edge[2]):
                        best_edge = (u, v, weight)
        if best_edge is None:
            break
        u, v, weight = best_edge
        visited_nodes.add(v)
        tree_edges.append((u, v, weight))
        total_cost += weight
        step_log.append(
            (u, v, weight, 'ACEITA',
             f'adiciona no {v} via ({u},{v})')
        )
    return tree_edges, total_cost, step_log

def mst_log_df(log_kruskal: list, log_prim: list) -> None:
    """Exibe os logs de Kruskal e Prim como DataFrames."""
    def to_dataframe(log: list) -> pd.DataFrame:
        return pd.DataFrame([
            {
                'Iter.': iteration,
                'Aresta': f'({u},{v})',
                'Peso': weight,
                'Decisao': decision,
            }
        ])

```

```

        'Obs.': observation,
    }
    for iteration, (u, v, weight, decision, observation)
    in enumerate(log, 1)
    ])

print('--- Kruskal ---')
display(to_dataframe(log_kruskal))
print('--- Prim ---')
display(to_dataframe(log_prim))

print('Algoritmos Kruskal e Prim carregados.')

```

Algoritmos Kruskal e Prim carregados.

```

In [95]: # =====
# Algoritmo Edmonds-Karp (BFS no grafo residual)
# =====
def edmonds_karp(
    nodes: list,
    arcs: list,
    source: int,
    sink: int,
) -> tuple:
    """Fluxo máximo pelo método de Edmonds-Karp.
    Retorna (fluxo_maximo, dict_fluxo_positivo, passos).
    """

    capacity = {}
    adjacency = {node: set() for node in nodes}
    for u, v, cap_val in arcs:
        capacity[(u, v)] = capacity.get((u, v), 0) + cap_val
        capacity.setdefault((v, u), 0)
        adjacency[u].add(v)
        adjacency[v].add(u)

    arc_flow = {arc: 0 for arc in capacity}
    augment_steps = []
    total_flow = 0

    while True:
        # BFS para encontrar caminho aumentante
        parent_map = {source: None}
        bfs_queue = deque([source])
        while bfs_queue:
            u = bfs_queue.popleft()
            if u == sink:
                break
            for v in adjacency[u]:
                residual = capacity[(u, v)] - arc_flow[(u, v)]
                if v not in parent_map and residual > 1e-9:
                    parent_map[v] = u
                    bfs_queue.append(v)

        if sink not in parent_map:
            break # sem caminho aumentante

        # reconstrói caminho e calcula gargalo
        augmenting_path, v = [], sink
        while parent_map[v] is not None:

```

```

        augmenting_path.append((parent_map[v], v))
        v = parent_map[v]
    augmenting_path.reverse()

    bottleneck = min(
        capacity[arc] - arc_flow[arc]
        for arc in augmenting_path
    )
    for u, v in augmenting_path:
        arc_flow[(u, v)] += bottleneck
        arc_flow[(v, u)] -= bottleneck
    total_flow += bottleneck

    path_nodes = [source] + [v for _, v in augmenting_path]
    path_str = ' -> '.join(str(n) for n in path_nodes)
    augment_steps.append({
        'Caminho': path_str,
        'Gargalo': int(bottleneck),
        'Fluxo acum.': int(total_flow),
    })

    positive_flow = {
        arc: flow
        for arc, flow in arc_flow.items()
        if flow > 0
    }
    return total_flow, positive_flow, augment_steps

print('Algoritmo Edmonds-Karp carregado.')

```

Algoritmo Edmonds-Karp carregado.

```

In [96]: # =====
# Visualização de grafos (AGM e fluxo máximo)
# =====
def draw_graph(
    nodes: list,
    edges: list,
    pos: dict,
    title: str,
    tree: list | None = None,
    directed: bool = False,
    flow: dict | None = None,
) -> None:
    """Desenha o grafo com destaques de AGM ou fluxo."""
    graph = nx.DiGraph() if directed else nx.Graph()
    for edge in edges:
        graph.add_edge(edge[0], edge[1], w=edge[2])

    mst_edge_set = set()
    if tree:
        mst_edge_set = {
            frozenset((u, v)) for u, v, _ in tree
        }

    edge_colors: list = []
    edge_widths: list = []
    for u, v in graph.edges():
        if frozenset((u, v)) in mst_edge_set:

```

```

        edge_colors.append('#d62728')
        edge_widths.append(3.2)
    else:
        edge_colors.append('#bbbbbb')
        edge_widths.append(1.3)

fig, ax = plt.subplots(figsize=(10, 7))
nx.draw_networkx_edges(
    graph, pos, ax=ax,
    edge_color=edge_colors,
    width=edge_widths,
    arrows=directed,
    arrowsize=18,
)
nx.draw_networkx_nodes(
    graph, pos, ax=ax,
    node_color='#2ec4c4',
    node_size=650,
    edgecolors='black',
)
nx.draw_networkx_labels(
    graph, pos, ax=ax,
    font_size=10, font_weight='bold',
)

if flow is not None:
    edge_labels = {}
    for u, v, attr in graph.edges(data=True):
        flow_val = int(flow.get((u, v), 0))
        edge_labels[(u, v)] = f'{flow_val}/{attr["w"]}'
else:
    edge_labels = {
        (u, v): attr['w']
        for u, v, attr in graph.edges(data=True)
    }

nx.draw_networkx_edge_labels(
    graph, pos,
    edge_labels=edge_labels,
    ax=ax,
    font_size=7,
    label_pos=0.5,
    bbox=dict(
        boxstyle='round,pad=0.1', fc='white', ec='none'
    ),
)
ax.set_title(title, fontweight='bold')
ax.axis('off')
plt.tight_layout()
plt.show()

print('Funcao draw_graph carregada.')

```

Funcao draw_graph carregada.

Exercício 1: Mínima Árvore Geradora (12 nós)

Grafo não-dirigido com 12 nós e 19 arestas. Custo total esperado = 74.

```
In [97]: EX1_EDGES = [
    (11,7,6),(11,12,11),(12,10,13),(7,6,22),(7,10,9),
    (6,3,5),(6,5,7),(3,1,7),(3,5,8),(3,4,6),
    (10,8,4),(10,9,7),(8,9,7),(8,5,10),(5,9,20),(5,4,9),
    (9,4,8),(4,2,4),(1,2,10),
]
EX1_POS = {
    11:(0.6,6.2), 7:(1.7,5.3), 12:(0.0,5.0), 6:(3.2,4.4),
    10:(1.4,3.7), 3:(4.2,2.9), 5:(3.0,2.4), 8:(1.4,1.9),
    9:(2.3,1.5), 1:(5.2,2.1), 4:(3.4,1.0), 2:(4.2,0.3),
}

n1 = list(range(1, 13))
tree_k1, z_k1, log_k1 = kruskal(n1, EX1_EDGES)
tree_p1, z_p1, log_p1 = prim(n1, EX1_EDGES, start=1)

print(f'Kruskal: custo = {z_k1} | Prim: custo = {z_p1}')
mst_log_df(log_k1, log_p1)
```

```
Kruskal: custo = 74 | Prim: custo = 74
--- Kruskal ---
```

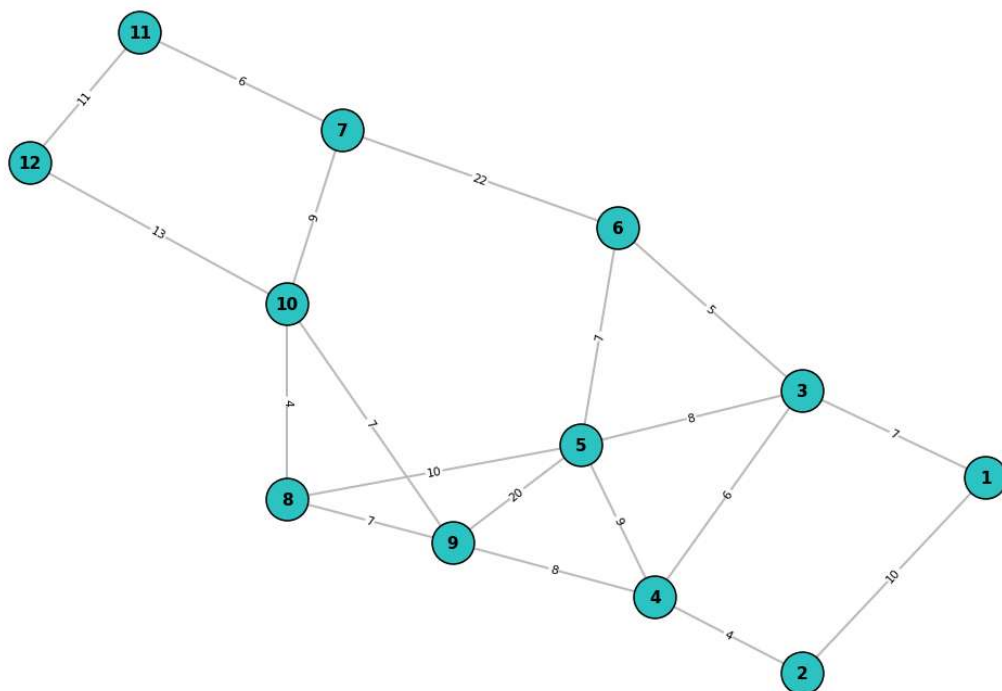
	Iter.	Aresta	Peso	Decisao	Obs.
0	1	(10,8)	4	ACEITA	une {10} e {8}
1	2	(4,2)	4	ACEITA	une {4} e {2}
2	3	(6,3)	5	ACEITA	une {6} e {3}
3	4	(11,7)	6	ACEITA	une {11} e {7}
4	5	(3,4)	6	ACEITA	une {3} e {4}
5	6	(6,5)	7	ACEITA	une {6} e {5}
6	7	(3,1)	7	ACEITA	une {3} e {1}
7	8	(10,9)	7	ACEITA	une {10} e {9}
8	9	(8,9)	7	rejeita	formaria ciclo
9	10	(3,5)	8	rejeita	formaria ciclo
10	11	(9,4)	8	ACEITA	une {9} e {4}
11	12	(7,10)	9	ACEITA	une {7} e {10}
12	13	(5,4)	9	rejeita	formaria ciclo
13	14	(8,5)	10	rejeita	formaria ciclo
14	15	(1,2)	10	rejeita	formaria ciclo
15	16	(11,12)	11	ACEITA	une {11} e {12}
16	17	(12,10)	13	rejeita	formaria ciclo
17	18	(5,9)	20	rejeita	formaria ciclo
18	19	(7,6)	22	rejeita	formaria ciclo

```
--- Prim ---
```

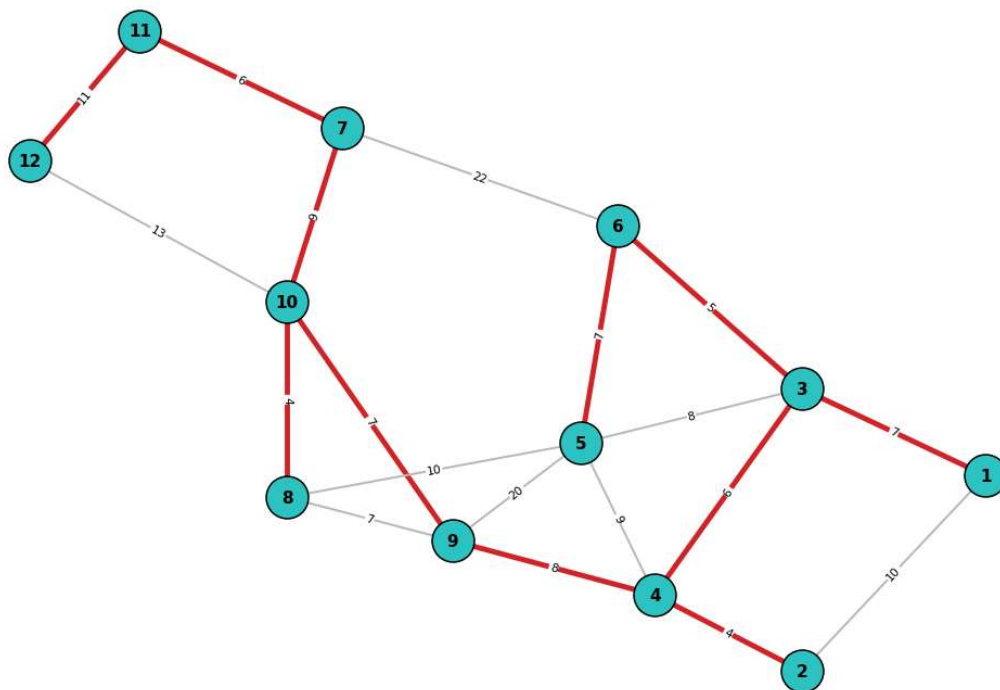
Iter.	Aresta	Peso	Decisao	Obs.
0	1 (1,3)	7	ACEITA	adiciona no 3 via (1,3)
1	2 (3,6)	5	ACEITA	adiciona no 6 via (3,6)
2	3 (3,4)	6	ACEITA	adiciona no 4 via (3,4)
3	4 (4,2)	4	ACEITA	adiciona no 2 via (4,2)
4	5 (6,5)	7	ACEITA	adiciona no 5 via (6,5)
5	6 (4,9)	8	ACEITA	adiciona no 9 via (4,9)
6	7 (9,10)	7	ACEITA	adiciona no 10 via (9,10)
7	8 (10,8)	4	ACEITA	adiciona no 8 via (10,8)
8	9 (10,7)	9	ACEITA	adiciona no 7 via (10,7)
9	10 (7,11)	6	ACEITA	adiciona no 11 via (7,11)
10	11 (11,12)	11	ACEITA	adiciona no 12 via (11,12)

```
In [98]: draw_graph(n1, EX1_EDGES, EX1_POS, 'Ex. 1 – Grafo de entrada')
draw_graph(n1, EX1_EDGES, EX1_POS,
           f'Ex. 1 – Mínima árvore geradora (custo total = {z_k1})',
           tree=tree_k1)
```

Ex. 1 – Grafo de entrada



Ex. 1 — Mínima árvore geradora (custo total = 74)



Exercício 2: Mínima Árvore Geradora (15 nós)

Grafo não-dirigido com 15 nós e 27 arestas. Custo total esperado = 89.

```
In [99]: EX2_EDGES = [
    (11,15,5),(11,12,11),(11,7,3),(12,7,6),(12,10,13),
    (15,14,7),(7,10,19),(7,14,3),(7,6,22),(14,6,7),(14,13,4),
    (10,8,14),(10,9,17),(10,5,10),(8,9,13),(9,5,11),
    (6,5,7),(6,3,9),(6,13,5),(5,3,8),(5,4,9),
    (3,13,5),(3,1,7),(3,4,6),(13,1,8),(4,2,4),(1,2,10),
]
EX2_POS = {
    11:(4.3,7.0),15:(5.6,6.9),12:(3.3,6.6), 7:(4.4,5.6),
    14:(6.2,5.4),10:(2.6,4.8), 6:(5.2,4.4), 8:(1.0,4.3),
    13:(7.2,4.4), 5:(3.6,3.9), 3:(5.0,3.6), 9:(2.0,3.5),
    4:(4.6,2.6), 1:(6.6,3.0), 2:(4.0,1.8),
}

n2 = list(range(1, 16))
tree_k2, z_k2, log_k2 = kruskal(n2, EX2_EDGES)
tree_p2, z_p2, log_p2 = prim(n2, EX2_EDGES, start=1)

print(f'Kruskal: custo = {z_k2} | Prim: custo = {z_p2}')
```

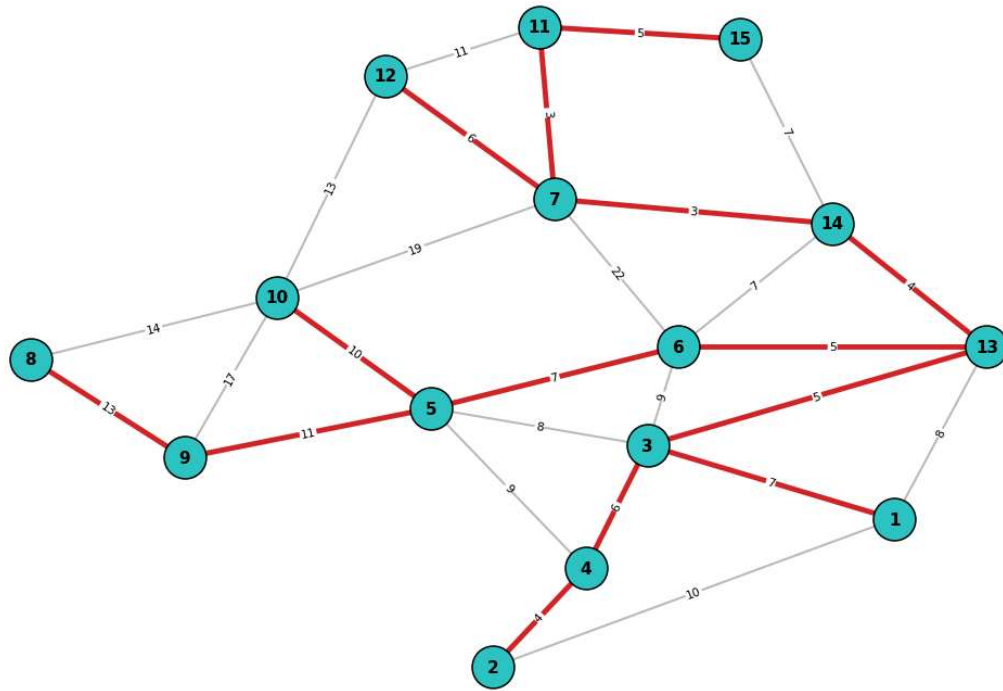
```
mst_log_df(log_k2, log_p2)

Kruskal: custo = 89 | Prim: custo = 89
--- Kruskal ---
```

	Iter.	Aresta	Peso	Decisao	Obs.
0	1	(11,7)	3	ACEITA	une {11} e {7}
1	2	(7,14)	3	ACEITA	une {7} e {14}
2	3	(14,13)	4	ACEITA	une {14} e {13}
3	4	(4,2)	4	ACEITA	une {4} e {2}
4	5	(11,15)	5	ACEITA	une {11} e {15}
5	6	(6,13)	5	ACEITA	une {6} e {13}
6	7	(3,13)	5	ACEITA	une {3} e {13}
7	8	(12,7)	6	ACEITA	une {12} e {7}
8	9	(3,4)	6	ACEITA	une {3} e {4}
9	10	(15,14)	7	rejeita	formaria ciclo
10	11	(14,6)	7	rejeita	formaria ciclo
11	12	(6,5)	7	ACEITA	une {6} e {5}
12	13	(3,1)	7	ACEITA	une {3} e {1}
13	14	(5,3)	8	rejeita	formaria ciclo
14	15	(13,1)	8	rejeita	formaria ciclo
15	16	(6,3)	9	rejeita	formaria ciclo
16	17	(5,4)	9	rejeita	formaria ciclo
17	18	(10,5)	10	ACEITA	une {10} e {5}
18	19	(1,2)	10	rejeita	formaria ciclo
19	20	(11,12)	11	rejeita	formaria ciclo
20	21	(9,5)	11	ACEITA	une {9} e {5}
21	22	(12,10)	13	rejeita	formaria ciclo
22	23	(8,9)	13	ACEITA	une {8} e {9}
23	24	(10,8)	14	rejeita	formaria ciclo
24	25	(10,9)	17	rejeita	formaria ciclo
25	26	(7,10)	19	rejeita	formaria ciclo
26	27	(7,6)	22	rejeita	formaria ciclo

--- Prim ---

Ex. 2 — Mínima árvore geradora (custo total = 89)



Exercício 5: Fluxo Máximo (Edmonds-Karp)

Grafo **dirigido** com 15 nós; fonte $s = 1$, sumidouro $t = 15$.

Modelo matemático:

$$\max F = \sum_{j:(1,j) \in A} f_{1j}$$

- Conservação: $\sum_{j:(i,j) \in A} f_{ij} - \sum_{j:(j,i) \in A} f_{ji} = 0 \quad \forall i \neq 1, 15$
- Capacidade: $0 \leq f_{ij} \leq u_{ij} \quad \forall (i, j) \in A$

O algoritmo de **Edmonds-Karp** utiliza BFS para encontrar o caminho aumentante de menor número de arestas a cada iteração.

In [101...

```
EX5_ARCS = [
    (1,2,13),(1,5,46),(1,9,40),(1,10,92),
    (2,3,27),(2,6,80),(2,14,88),
    (3,4,90),(3,7,79),(3,8,26),(3,11,37),(3,12,43),(3,13,42),
    (4,15,44),(5,3,82),(5,6,43),(5,14,46),
    (6,4,80),(6,7,56),(6,8,38),(6,11,74),(6,12,81),(6,13,35),
    (7,15,34),(8,15,63),
    (9,3,47),(9,6,77),(9,14,13),(10,3,19),(10,6,60),(10,14,70),
    (11,15,57),(12,15,74),(13,15,62),
    (14,4,63),(14,7,15),(14,8,59),(14,11,90),(14,12,74),(14,13,45),
]
EX5_POS = {
    7:(5.0,7.2),10:(2.8,6.0),13:(7.2,6.0), 5:(1.8,4.8),
    14:(4.6,4.9), 8:(7.2,4.9), 1:(0.6,3.8), 3:(5.0,3.8),
    15:(8.4,3.8), 9:(1.8,2.7), 6:(4.0,2.8), 4:(6.2,2.2),
    2:(2.4,1.3),11:(5.0,0.8),12:(6.6,1.0),
}
```

```

}

n5 = list(range(1, 16))
mf, pos_fl, steps = edmonds_karp(n5, EX5_ARCS, source=1, sink=15)

print(f'Fluxo máximo: {mf}')
print(f'Soma das capacidades da fonte: 13+46+40+92 = {13+46+40+92} (todos os arc
display(pd.DataFrame(steps))

```

Fluxo máximo: 191

Soma das capacidades da fonte: 13+46+40+92 = 191 (todos os arcos saturados)

	Caminho	Gargalo	Fluxo acum.
0	1 -> 9 -> 3 -> 4 -> 15	40	40
1	1 -> 2 -> 3 -> 4 -> 15	4	44
2	1 -> 2 -> 3 -> 7 -> 15	9	53
3	1 -> 10 -> 3 -> 7 -> 15	19	72
4	1 -> 10 -> 6 -> 7 -> 15	6	78
5	1 -> 10 -> 6 -> 8 -> 15	38	116
6	1 -> 10 -> 6 -> 11 -> 15	16	132
7	1 -> 10 -> 14 -> 8 -> 15	13	145
8	1 -> 5 -> 3 -> 8 -> 15	12	157
9	1 -> 5 -> 3 -> 11 -> 15	34	191

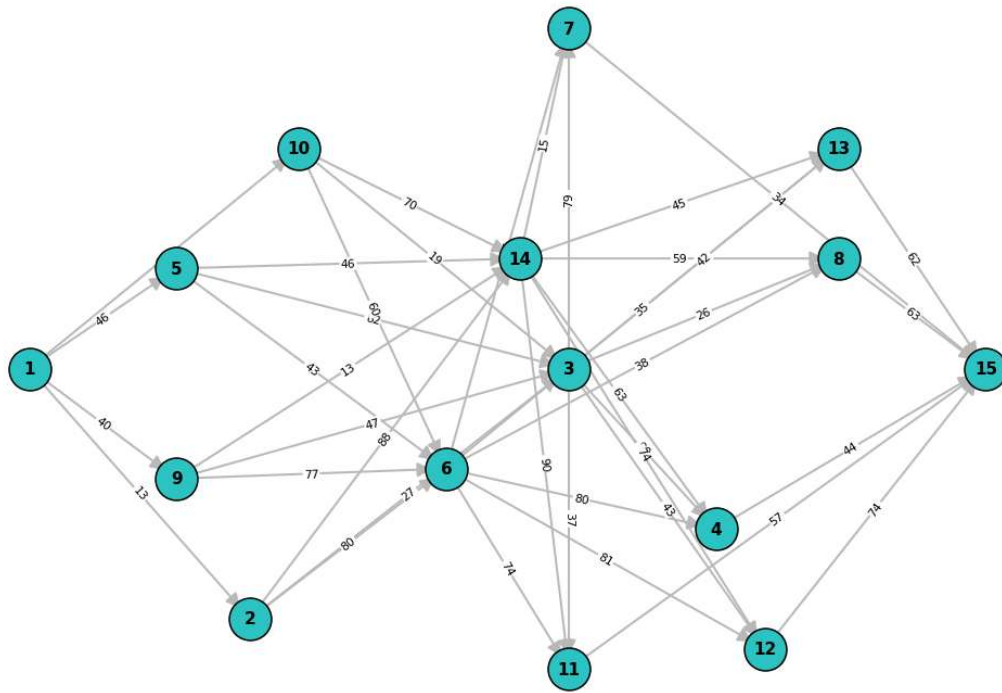
In [102...

```

draw_graph(n5, EX5_ARCS, EX5_POS,
            'Ex. 5 - Grafo dirigido (capacidades)',
            directed=True)
draw_graph(n5, EX5_ARCS, EX5_POS,
            f'Ex. 5 - Fluxo ótimo (fluxo/capacidade) | fluxo máximo = {mf}',
            directed=True, flow=pos_fl)

```

Ex. 5 — Grafo dirigido (capacidades)



Ex. 5 — Fluxo ótimo (fluxo/capacidade) | fluxo máximo = 191

